

Projet fil rouge — Blog Laravel

Exercice de conception · Travail individuel

1. Contexte

Dans le cadre de la formation DWWM, tu vas développer un blog de A à Z avec Laravel. Ce projet fil rouge accompagne toute la séquence backend : chaque notion abordée en cours — routing, Blade, Eloquent, authentification, middlewares — sera appliquée directement sur ce projet.

Avant d'écrire la moindre ligne de code, tu dois concevoir ce que tu vas construire. Un développeur qui code sans avoir modélisé au préalable accumule les incohérences, revient en arrière, et perd du temps. Cet exercice t'oblige à réfléchir d'abord.

Les livrables que tu produiras ici te serviront de référence tout au long du développement. Garde-les accessibles.

2. Description du projet

Le blog est une application web avec trois types d'acteurs :

- Un **visiteur non connecté** peut parcourir et lire le blog librement, sans compte.
- Un **utilisateur inscrit** peut commenter les articles et gérer ses propres commentaires.
- Un **administrateur** gère l'ensemble du contenu : articles, catégories, tags et commentaires.

3. Cahier des charges

Visiteur (non connecté)

- Voir la liste paginée des articles publiés (du plus récent au plus ancien)
- Filtrer la liste par catégorie
- Filtrer la liste par tag
- Combiner les filtres catégorie et tag simultanément
- Voir le détail d'un article : titre, contenu, catégorie, tags, commentaires
- Lire les commentaires d'un article

Utilisateur (connecté)

- S'inscrire avec un nom, un email et un mot de passe
- Se connecter / se déconnecter
- Poster un commentaire sur un article
- Modifier son propre commentaire
- Supprimer son propre commentaire

Administrateur

- Se connecter / se déconnecter
- Créer un article (statut : brouillon ou publié)
- Modifier un article
- Publier ou dépublier un article
- Supprimer un article
- Supprimer n'importe quel commentaire
- Créer, modifier et supprimer des catégories
- Créer, modifier et supprimer des tags

Règles métier

- Seuls les articles avec le statut "publié" sont visibles des visiteurs et utilisateurs. Les brouillons restent invisibles hors de l'interface d'administration.
- Un article appartient à exactement une catégorie, et peut porter plusieurs tags.
- Les commentaires sont visibles par tous — connectés ou non.
- Poster, modifier ou supprimer un commentaire nécessite d'être connecté.
- Un utilisateur ne peut modifier ou supprimer que ses propres commentaires.
- La liste des articles est paginée (valeur par défaut du framework) et triée du plus récent au plus ancien.

4. Ce qui est attendu

Produis les trois livrables suivants. Le format est libre — à la main, sur outil en ligne, peu importe — tant que c'est lisible et exportable en PDF ou image.

Livable	Ce qui est attendu	Outils suggérés
Use cases	Liste des cas d'utilisation par acteur. Pour chaque use case : nom, acteur, déclencheur, résultat attendu.	Tableau Word/PDF, draw.io, Excalidraw, papier scanné...
Maquettes (wireframes)	Fil de fer des écrans principaux : liste des articles, détail d'un article, formulaire de commentaire, formulaire de création/édition d'article (admin). Pas de design — les blocs et zones suffisent.	Excalidraw, Figma, Balsamiq, papier scanné...
Modélisation BDD	MCD puis MLD : toutes les entités, leurs attributs avec types de données, les relations et cardinalités. Le MLD doit être prêt à se traduire directement en migrations Laravel.	Looping, draw.io, dbdiagram.io, papier scanné...



Conseil : dans quel ordre travailler

1. Commence par les use cases — ils te forcent à lister toutes les actions possibles.
2. Les actions révèlent les entités dont tu as besoin (Article, User, Comment...).
3. Les entités et leurs relations te donnent le MCD, puis le MLD.
4. Le MLD te montre quelles données sont disponibles — tu peux alors maquetter les écrans en sachant quoi afficher.

Dans cet ordre, chaque livrable découle naturellement du précédent.

5. Hors scope

Les fonctionnalités suivantes ne font pas partie du projet. Ne les modélise pas, ne les développe pas.

- Upload d'images ou de médias sur les articles
- Système de notifications
- Moteur de recherche plein texte
- Réponses imbriquées aux commentaires (threads)

📌 Ces fonctionnalités sont des extensions naturelles et réalistes du projet. Si tu veux les explorer en dehors des séances, libre à toi — mais le périmètre de la formation s'arrête au scope ci-dessus.

6. Rendu

Tes livrables sont à remettre avant de démarrer le code. Tu recevras ensuite une correction de référence pour t'assurer que ta modélisation est cohérente avant de l'implémenter.

Organisation dans le repo Git

Tu vas créer un repo GitHub dédié à ce projet : [DWWM-blog](#). Ce repo hébergera à la fois ton code Laravel et tes documents de conception. Crée un dossier [docs/](#) à la racine pour y déposer tes livrables :

```
DWWM-blog/
├── docs/
│   ├── use-cases.pdf    ← ou .png, .md, peu importe le format
│   ├── wireframes.pdf
│   └── modelisation.pdf
├── app/                ← code Laravel (plus tard)
└── ...
```

Pratique Git attendue

Travaille avec Git dès la conception — pas uniquement quand tu codes. Quelques règles à appliquer :

- **Committe fréquemment** : un commit dès que tu termines une partie identifiable (un use case, un écran, le MCD...).
- **Commits atomiques** : un commit = une chose. Ne mélange pas "ajout du MCD" et "correction du use case 3" dans le même commit.
- **Messages explicites** : le message doit dire ce qui a été fait, pas juste "update" ou "modif". Utilise les préfixes conventionnels appris en cours.

💡 **Pourquoi committer ses documents de conception ?** Ton historique Git raconte l'évolution de ta réflexion, pas seulement de ton code. Si ta modélisation change en cours de route, tu gardes une trace de la version précédente. C'est aussi ce qu'on fait en équipe professionnelle : la conception vit dans le repo, pas dans un coin de bureau.

⚠️ **Important** Une modélisation incohérente se répercute sur tout le code qui suit — mieux vaut la corriger avant de passer au code. En cas de doute sur un point du cahier des charges, pose la question avant de modéliser.